Complete API Integration Guide

AI Pre-Placement Trainer — Wikipedia API + YouTube Data API v3

Industry-Level Implementation with Full Error Handling

Project: AI-Based Pre-Placement Trainer and Feedback Model Institution: YSPM's Yashoda Technical Campus, Satara Department: AI & Data Science | Academic Year 2025-26

PART 1 — WIKIPEDIA API

1.1 What is the Wikipedia API and Why Use It

The Wikipedia REST API and MediaWiki Action API are completely free, require no API key, have no daily quota limits, and are licensed under Creative Commons Attribution-ShareAlike (CC BY-SA), which means you can legally use the content in your educational project without any copyright issues. This makes it the perfect solution for fetching rich aptitude learning content for topics like Number Systems, Percentages, Time and Work, Probability, and all other topics in your aptitude module.

When a student clicks on a topic in your Learning section, instead of showing hardcoded short HTML text from aptitudeData.ts, your system will fetch a properly structured Wikipedia summary and introduction, display it as theory content, and cache it in your database so it never hits Wikipedia again for the same topic. This gives you unlimited, always-updated, legally usable content for every aptitude topic without writing a single paragraph yourself.

1.2 Wikipedia API — No Setup Required

Unlike the YouTube API, the Wikipedia API requires absolutely no registration, no API key, no Google Cloud Console, and no billing account. You simply make HTTP requests to their public endpoints. The only requirement is that you add a User-Agent header to your requests identifying your application, which Wikipedia requires by policy so they can contact you if there is a problem.

There is no dashboard to visit, no credentials to create, and no quota to worry about. Wikipedia's servers handle billions of requests per day and their API is designed for developers exactly like you.

1.3 The Two Wikipedia API Endpoints You Will Use

Endpoint 1 — Wikipedia REST API (Summary)

This is the modern, simple endpoint. It returns a clean JSON object with the page title, a short summary paragraph, an extract of the full introduction, and a thumbnail image if available.

```
https://en.wikipedia.org/api/rest_v1/page/summary/{ARTICLE_TITLE}
```

Example for Percentages:

```
https://en.wikipedia.org/api/rest_v1/page/summary/Percentage
```

This returns JSON like:


```
{
  "title": "Percentage",
  "displaytitle": "Percentage",
  "description": "Number or ratio expressed as a fraction of 100",
  "extract": "In mathematics, a percentage is a number or ratio expressed
as a fraction of 100. It is often denoted using the percent sign, %,
although the abbreviations pct., pct and sometimes pc are also used. A
percentage is a dimensionless number; it has no unit of measurement.",
  "extract_html": "<p>In mathematics, a <b>percentage</b> is a
number...</p>",
  "thumbnail": {
    "source": "https://upload.wikimedia.org/...",
    "width": 320,
    "height": 240
  },
  "content_urls": {
    "desktop": {
      "page": "https://en.wikipedia.org/wiki/Percentage"
    }
  }
}
```


Endpoint 2 — MediaWiki Action API (Full Sections)

This endpoint gives you more control. You can fetch specific sections of a Wikipedia article, search for the best matching article by keyword, and get the full introduction text with all subsections.

```
https://en.wikipedia.org/w/api.php?action=query&titles={TITLE}&prop=extract
s&exintro=true&explaintext=true&format=json
```

You will use both endpoints in your project. The REST API for quick summaries and the Action API for fetching richer, longer content when the student clicks "Read Full Theory."

1.4 Topic-to-Wikipedia Mapping for Your Aptitude Module

This is a critical part of the implementation. You need to map each of your aptitude topic names to the correct Wikipedia article title. Your topic names in the database might be "Time and Work" but Wikipedia calls it "Time and work (mathematics)". The mapping table below covers all standard aptitude topics:

```
# apps/aptitude/wikipedia_mapping.py
# This file maps your internal topic names to Wikipedia article titles

TOPIC_WIKIPEDIA_MAP = {
    # Quantitative Aptitude
    "Number Systems": "Number",
    "Numbers": "Number",
    "HCF and LCM": "Least common multiple",
    "Percentages": "Percentage",
    "Profit and Loss": "Profit (economics)",
    "Simple Interest": "Simple interest",
    "Compound Interest": "Compound interest",
    "Time and Work": "Time and work (mathematics)",
    "Time Speed Distance": "Speed",
    "Pipes and Cisterns": "Pipe (fluid conveyance)",
    "Ratio and Proportion": "Ratio",
    "Averages": "Average",
    "Ages": "Age (mathematics)",
    "Probability": "Probability",
    "Permutations and Combinations": "Permutation",
    "Algebra": "Algebra",
    "Quadratic Equations": "Quadratic equation",
    "Geometry": "Geometry",
    "Mensuration": "Measurement",
    "Trigonometry": "Trigonometry",
    "Data Interpretation": "Data analysis",
    "Logarithms": "Logarithm",
    "Sets and Functions": "Set (mathematics)",

    # Logical Reasoning
    "Logical Reasoning": "Deductive reasoning",
    "Blood Relations": "Kinship",
    "Direction Sense": "Cardinal direction",
    "Coding and Decoding": "Cipher",
    "Seating Arrangement": "Combinatorics",
    "Syllogism": "Syllogism",
    "Series Completion": "Sequence",
    "Analogies": "Analogy",
    "Puzzles": "Puzzle",
    "Statement and Conclusion": "Logical consequence",

    # Verbal Ability
    "Verbal Ability": "Linguistics",
    "Reading Comprehension": "Reading comprehension",
    "Synonyms and Antonyms": "Synonym",
    "Grammar": "English grammar",
    "Sentence Correction": "English grammar",
    "Vocabulary": "Vocabulary",
    "Para Jumbles": "Paragraph",
```

```
    # Default fallback
    "default": "Mathematics"
}
```

1.5 Backend Implementation — Django Service File

Create a new file at this exact path in your project:

```
backend/apps/aptitude/services/wikipedia_service.py
```

Copy the complete code below into that file:

```
# backend/apps/aptitude/services/wikipedia_service.py
"""
Wikipedia API Service for AI Pre-Placement Trainer
Fetches aptitude learning content from Wikipedia REST API and Action API.
All content is legally usable under Creative Commons Attribution-ShareAlike
license.
Author: Based on project blueprint by Vedant Kumbhar, Meeraj Krishna,
Loukik Ingale
"""

import requests
import logging
from typing import Optional, Dict, Any
from django.core.cache import cache

# Set up logging so errors appear in your Django logs
logger = logging.getLogger(__name__)

# Your application identifier — Wikipedia requires this in the User-Agent
header
USER_AGENT = "AI-PrePlacement-Trainer/1.0 (YSPM Yashoda Technical Campus;
academic project)"

# Cache timeout — 7 days (604800 seconds)
# Wikipedia content does not change frequently, so 7 days is safe
CACHE_TIMEOUT = 604800

# Request timeout — if Wikipedia takes more than 8 seconds, give up
REQUEST_TIMEOUT = 8


class WikipediaServiceError(Exception):
    """
    Custom exception for Wikipedia service failures.
    Raised when content cannot be fetched and no cache exists.
    """
    pass


class WikipediaService:
    """
    Service class that wraps Wikipedia REST API and Action API calls.
    Handles caching, retries, error handling, and content cleaning.
    """
```

```
    # Base URLs for both Wikipedia API endpoints
    REST_API_BASE = "https://en.wikipedia.org/api/rest_v1"
    ACTION_API_BASE = "https://en.wikipedia.org/w/api.php"

    # Standard headers required by Wikipedia API policy
    HEADERS = {
        "User-Agent": USER_AGENT,
        "Accept": "application/json",
        "Accept-Language": "en-US,en;q=0.9",
    }

    def get_topic_summary(self, article_title: str) -> Dict[str, Any]:
        """
        Fetch a short summary of a Wikipedia article using the REST API.
        Returns title, description, 2-3 paragraph extract, and thumbnail
URL.

        Args:
            article_title: The Wikipedia article title (e.g., "Percentage")

        Returns:
            Dictionary with keys: title, description, extract,
extract_html,
            thumbnail_url, wikipedia_url, source

        Raises:
            WikipediaServiceError: When content cannot be fetched
        """
        # Check cache first — avoid hitting Wikipedia on every request
        cache_key = f"wiki_summary_{article_title.replace(' ',
'_').lower()}"
        cached_data = cache.get(cache_key)

        if cached_data:
            logger.info(f"Wikipedia summary cache HIT for:
{article_title}")
            return cached_data

        logger.info(f"Wikipedia summary cache MISS for: {article_title} —
fetching from API")

        # Replace spaces with underscores for the URL
        url_title = article_title.replace(" ", "_")
        url = f"{self.REST_API_BASE}/page/summary/{url_title}"

        try:
            response = requests.get(
                url,
                headers=self.HEADERS,
                timeout=REQUEST_TIMEOUT
            )

            # Handle HTTP error codes explicitly
            if response.status_code == 404:
                logger.warning(f"Wikipedia article not found:
{article_title}")
                raise WikipediaServiceError(
                    f"No Wikipedia article found for '{article_title}'. "
                    f"Try a different search term."
                )
```

```
            if response.status_code == 301:
                # Wikipedia redirected to a different article title
                # The response still contains the correct data, so continue
                logger.info(f"Wikipedia redirected for: {article_title}")

            if response.status_code not in [200, 301]:
                logger.error(
                    f"Wikipedia REST API returned status
{response.status_code} "
                    f"for article: {article_title}"
                )
                raise WikipediaServiceError(
                    f"Wikipedia API returned error {response.status_code}.
"
                    f"Please try again later."
                )

            data = response.json()

            # Build a clean, structured result
            result = {
                "title": data.get("title", article_title),
                "display_title": data.get("displaytitle", article_title),
                "description": data.get("description", ""),
                "extract": data.get("extract", ""),
                "extract_html": data.get("extract_html", ""),
                "thumbnail_url": (
                    data.get("thumbnail", {}).get("source", None)
                ),
                "wikipedia_url": (
                    data.get("content_urls", {})
                    .get("desktop", {})
                    .get("page",
f"https://en.wikipedia.org/wiki/{url_title}")
                ),
                "source": "Wikipedia",
                "license": "CC BY-SA 4.0",
                "from_cache": False,
            }

            # Store in cache for 7 days
            cache.set(cache_key, result, CACHE_TIMEOUT)

            return result

        except requests.exceptions.Timeout:
            logger.error(f"Wikipedia API timed out for article:
{article_title}")
            raise WikipediaServiceError(
                "Wikipedia is taking too long to respond. "
                "Please try again in a few seconds."
            )

        except requests.exceptions.ConnectionError:
            logger.error(f"Could not connect to Wikipedia for article:
{article_title}")
            raise WikipediaServiceError(
                "Could not connect to Wikipedia. "
                "Please check your internet connection."
            )
```

```
        except requests.exceptions.JSONDecodeError:
            logger.error(f"Wikipedia returned invalid JSON for article:
{article_title}")
            raise WikipediaServiceError(
                "Wikipedia returned an unexpected response. Please try
again."
            )

        except Exception as e:
            logger.error(
                f"Unexpected error fetching Wikipedia article
'{article_title}': {str(e)}"
            )
            raise WikipediaServiceError(
                f"An unexpected error occurred while fetching content. "
                f"Please try again later."
            )

    def get_topic_full_content(self, article_title: str) -> Dict[str, Any]:
        """
        Fetch full introduction section of a Wikipedia article using the
Action API.
        Returns much more content than get_topic_summary — suitable for the
        full theory page in your Learning module.

        Args:
            article_title: The Wikipedia article title

        Returns:
            Dictionary with keys: title, full_extract, sections, source

        Raises:
            WikipediaServiceError: When content cannot be fetched
        """
        cache_key = f"wiki_full_{article_title.replace(' ', '_').lower()}"
        cached_data = cache.get(cache_key)

        if cached_data:
            logger.info(f"Wikipedia full content cache HIT for:
{article_title}")
            return cached_data

        logger.info(
            f"Wikipedia full content cache MISS for: {article_title} —
fetching from API"
        )

        params = {
            "action": "query",
            "titles": article_title,
            "prop": "extracts",
            "exintro": True,        # Only the introduction section
            "explaintext": True,    # Plain text, not HTML
            "exsectionformat": "plain",
            "exsentences": 20,      # Maximum 20 sentences
            "format": "json",
            "origin": "*",          # Required for CORS
        }

        try:
            response = requests.get(
```

```
                self.ACTION_API_BASE,
                params=params,
                headers=self.HEADERS,
                timeout=REQUEST_TIMEOUT
            )

            if response.status_code != 200:
                logger.error(
                    f"Wikipedia Action API returned status
{response.status_code} "
                    f"for: {article_title}"
                )
                raise WikipediaServiceError(
                    f"Wikipedia API error: {response.status_code}"
                )

            data = response.json()

            # The Action API returns pages in a nested structure
            pages = data.get("query", {}).get("pages", {})

            if not pages:
                raise WikipediaServiceError(
                    f"No content found for '{article_title}' on Wikipedia."
                )

            # Get the first (and usually only) page
            page_id = list(pages.keys())[0]

            # page_id of -1 means the article does not exist
            if page_id == "-1":
                logger.warning(f"Wikipedia article missing:
{article_title}")
                raise WikipediaServiceError(
                    f"Wikipedia article '{article_title}' does not exist. "
                    f"Please check the topic name."
                )

            page_data = pages[page_id]
            extract = page_data.get("extract", "")

            # Clean up the extract text
            extract = self._clean_extract(extract)

            result = {
                "title": page_data.get("title", article_title),
                "full_extract": extract,
                "word_count": len(extract.split()),
                "source": "Wikipedia",
                "license": "CC BY-SA 4.0",
                "wikipedia_url": (
                    f"https://en.wikipedia.org/wiki/"
                    f"{article_title.replace(' ', '_')}"
                ),
                "from_cache": False,
            }

            # Cache for 7 days
            cache.set(cache_key, result, CACHE_TIMEOUT)

            return result
```

```
        except WikipediaServiceError:
            # Re-raise our own errors without wrapping them again
            raise

        except requests.exceptions.Timeout:
            logger.error(
                f"Wikipedia Action API timed out for: {article_title}"
            )
            raise WikipediaServiceError(
                "Request to Wikipedia timed out. Please try again."
            )

        except requests.exceptions.ConnectionError:
            logger.error(
                f"Connection error accessing Wikipedia for:
{article_title}"
            )
            raise WikipediaServiceError(
                "Cannot connect to Wikipedia. Check your internet
connection."
            )

        except Exception as e:
            logger.error(
                f"Unexpected error in get_topic_full_content "
                f"for '{article_title}': {str(e)}"
            )
            raise WikipediaServiceError(
                "An unexpected error occurred. Please try again later."
            )

    def search_wikipedia(self, query: str, limit: int = 5) -> list:
        """
        Search Wikipedia for articles matching a query.
        Used as a fallback when the direct article title lookup fails.

        Args:
            query: Search query string (e.g., "time and work aptitude")
            limit: Maximum number of results to return (default 5, max 10)

        Returns:
            List of dictionaries with keys: title, snippet, url
        """
        cache_key = f"wiki_search_{query.replace(' ', '_').lower()}"
        cached_data = cache.get(cache_key)

        if cached_data:
            return cached_data

        params = {
            "action": "query",
            "list": "search",
            "srsearch": query,
            "srlimit": min(limit, 10),
            "srinfo": "totalhits",
            "srprop": "snippet|titlesnippet",
            "format": "json",
        }

        try:
```

```
            response = requests.get(
                self.ACTION_API_BASE,
                params=params,
                headers=self.HEADERS,
                timeout=REQUEST_TIMEOUT
            )

            if response.status_code != 200:
                logger.warning(f"Wikipedia search failed for query:
{query}")
                return []

            data = response.json()
            search_results = data.get("query", {}).get("search", [])

            results = [
                {
                    "title": item["title"],
                    "snippet": item.get("snippet", ""),
                    "url":
f"https://en.wikipedia.org/wiki/{item['title'].replace(' ', '_')}",
                }
                for item in search_results
            ]

            cache.set(cache_key, results, CACHE_TIMEOUT)
            return results

        except Exception as e:
            logger.error(f"Wikipedia search error for '{query}': {str(e)}")
            return []

    def get_content_for_topic(
        self, topic_name: str, wikipedia_mapping: dict
    ) -> Dict[str, Any]:
        """
        Main method called from your Django views.
        Tries direct lookup first, falls back to search if not found.
        Always returns a valid response — never crashes the view.

        Args:
            topic_name: Your internal topic name (e.g., "Time and Work")
            wikipedia_mapping: The TOPIC_WIKIPEDIA_MAP dictionary

        Returns:
            Dictionary with content data, or a fallback response on failure
        """
        # Step 1: Look up the mapped Wikipedia article title
        article_title = wikipedia_mapping.get(
            topic_name,
            wikipedia_mapping.get("default", topic_name)
        )

        logger.info(
            f"Fetching Wikipedia content for topic '{topic_name}' "
            f"using article title '{article_title}'"
        )

        # Step 2: Try fetching the summary
        try:
            summary = self.get_topic_summary(article_title)
```

```
            full_content = self.get_topic_full_content(article_title)

            return {
                "success": True,
                "topic_name": topic_name,
                "article_title": article_title,
                "summary": summary.get("extract", ""),
                "summary_html": summary.get("extract_html", ""),
                "full_content": full_content.get("full_extract", ""),
                "description": summary.get("description", ""),
                "thumbnail_url": summary.get("thumbnail_url"),
                "wikipedia_url": summary.get("wikipedia_url"),
                "source": "Wikipedia",
                "license": "CC BY-SA 4.0",
                "error": None,
            }

        except WikipediaServiceError as e:
            logger.warning(
                f"Direct lookup failed for '{article_title}', "
                f"attempting search fallback for '{topic_name}': {str(e)}"
            )

            # Step 3: Fallback — search Wikipedia for the topic
            search_results = self.search_wikipedia(
                f"{topic_name} mathematics aptitude"
            )

            if search_results:
                fallback_title = search_results[0]["title"]
                logger.info(f"Using fallback article: {fallback_title}")

                try:
                    summary = self.get_topic_summary(fallback_title)
                    return {
                        "success": True,
                        "topic_name": topic_name,
                        "article_title": fallback_title,
                        "summary": summary.get("extract", ""),
                        "summary_html": summary.get("extract_html", ""),
                        "full_content": summary.get("extract", ""),
                        "description": summary.get("description", ""),
                        "thumbnail_url": summary.get("thumbnail_url"),
                        "wikipedia_url": summary.get("wikipedia_url"),
                        "source": "Wikipedia",
                        "license": "CC BY-SA 4.0",
                        "error": None,
                        "used_fallback": True,
                    }
                except WikipediaServiceError:
                    pass

            # Step 4: Final fallback — return empty content with error flag
            logger.error(
                f"All Wikipedia fetch attempts failed for topic:
{topic_name}"
            )
            return {
                "success": False,
                "topic_name": topic_name,
                "article_title": article_title,
```

```
                "summary": "",
                "summary_html": "",
                "full_content": "",
                "description": "",
                "thumbnail_url": None,
                "wikipedia_url": (
                    f"https://en.wikipedia.org/wiki/"
                    f"{article_title.replace(' ', '_')}"
                ),
                "source": "Wikipedia",
                "license": "CC BY-SA 4.0",
                "error": str(e),
            }

    def _clean_extract(self, text: str) -> str:
        """
        Clean up Wikipedia extract text.
        Removes excessive whitespace, citation markers, and formatting
artifacts.

        Args:
            text: Raw extract text from Wikipedia API

        Returns:
            Cleaned text string
        """
        if not text:
            return ""

        # Remove multiple consecutive newlines
        import re
        text = re.sub(r"\n{3,}", "\n\n", text)

        # Remove lines that are just "== Section Title =="
        text = re.sub(r"^==+.*?==+$", "", text, flags=re.MULTILINE)

        # Remove citation-style markers like [1], [2], [citation needed]
        text = re.sub(r"\[\d+\]", "", text)
        text = re.sub(r"\[citation needed\]", "", text,
flags=re.IGNORECASE)

        # Remove excessive spaces
        text = re.sub(r" {2,}", " ", text)

        # Strip leading and trailing whitespace
        text = text.strip()

        return text
```

1.6 Django View for Wikipedia Content

Add this to your existing `apps/aptitude/views.py` file:

```
# Add these imports at the top of apps/aptitude/views.py
from .services.wikipedia_service import WikipediaService,
WikipediaServiceError
from .wikipedia_mapping import TOPIC_WIKIPEDIA_MAP
```

```
# Initialize the service once at module level
wikipedia_service = WikipediaService()


@api_view(['GET'])
@permission_classes([IsAuthenticated])
def get_topic_wikipedia_content(request, topic_id):
    """
    Fetch Wikipedia learning content for a specific aptitude topic.
    Called when student clicks a topic in the Learning sidebar.

    URL: GET /api/aptitude/topics/{topic_id}/wiki-content/

    Response on success:
    {
        "success": true,
        "topic_id": 1,
        "topic_name": "Percentages",
        "summary": "In mathematics, a percentage is...",
        "summary_html": "<p>In mathematics, a <b>percentage</b> is...</p>",
        "full_content": "A percentage is a number or ratio...",
        "description": "Number or ratio expressed as a fraction of 100",
        "thumbnail_url": "https://upload.wikimedia.org/...",
        "wikipedia_url": "https://en.wikipedia.org/wiki/Percentage",
        "source": "Wikipedia",
        "license": "CC BY-SA 4.0"
    }

    Response on failure (still 200 — never crash the learning page):
    {
        "success": false,
        "topic_name": "Some Topic",
        "error": "No Wikipedia article found for this topic.",
        "summary": "",
        ...
    }
    """
    try:
        topic = AptitudeTopic.objects.get(id=topic_id, is_active=True)
    except AptitudeTopic.DoesNotExist:
        return Response(
            {"error": f"Topic with ID {topic_id} not found."},
            status=404
        )

    # Fetch Wikipedia content using the service
    content = wikipedia_service.get_content_for_topic(
        topic_name=topic.name,
        wikipedia_mapping=TOPIC_WIKIPEDIA_MAP
    )

    # Always include the topic_id in the response
    content["topic_id"] = topic_id

    # If the topic has its own custom HTML content in the database,
    # merge it with the Wikipedia content
    if topic.learning_html:
        content["custom_html"] = topic.learning_html
        content["has_custom_content"] = True
    else:
        content["has_custom_content"] = False
```

```
    # Return 200 even on Wikipedia failure — the learning page should
    # still load, just without the Wikipedia section
    return Response(content, status=200)


@api_view(['GET'])
@permission_classes([IsAuthenticated])
def search_wikipedia_for_topic(request):
    """
    Search Wikipedia for articles matching a custom query.
    Used when student types in the search box on the learning page.

    URL: GET /api/aptitude/wiki-search/?q=probability+mathematics

    Query params:
        q (required): Search query string
        limit (optional): Number of results, default 5, max 10
    """
    query = request.query_params.get('q', '').strip()

    if not query:
        return Response(
            {"error": "Search query 'q' parameter is required."},
            status=400
        )

    if len(query) < 3:
        return Response(
            {"error": "Search query must be at least 3 characters."},
            status=400
        )

    limit = int(request.query_params.get('limit', 5))
    limit = min(max(limit, 1), 10)  # Clamp between 1 and 10

    results = wikipedia_service.search_wikipedia(query, limit)

    return Response({
        "query": query,
        "total_results": len(results),
        "results": results,
    }, status=200)
```

1.7 Add URLs for Wikipedia Endpoints

Add these lines to `apps/aptitude/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    # Existing URLs
    path('topics/', views.get_topics, name='aptitude-topics'),
    path('topics/<int:topic_id>/', views.get_topic_detail, name='aptitude-
topic-detail'),
    path('questions/', views.get_questions, name='aptitude-questions'),
    path('submit/', views.submit_quiz, name='aptitude-submit'),
```

```
    path('history/', views.get_quiz_history, name='aptitude-history'),

    # YouTube videos — from previous session
    path('topics/<int:topic_id>/videos/', views.get_topic_videos,
name='topic-videos'),

    # Wikipedia content — NEW
    path('topics/<int:topic_id>/wiki-content/',
views.get_topic_wikipedia_content, name='topic-wiki-content'),
    path('wiki-search/', views.search_wikipedia_for_topic, name='wiki-
search'),
]
```

1.8 Frontend — Wikipedia Content Component

Create this file at `src/components/learning/WikipediaContent.tsx`:

```
// src/components/learning/WikipediaContent.tsx
import { useState, useEffect } from 'react';
import { aptitudeAPI } from '../../services/aptitudeAPI';

interface WikiContent {
  success: boolean;
  topic_name: string;
  summary: string;
  summary_html: string;
  full_content: string;
  description: string;
  thumbnail_url: string | null;
  wikipedia_url: string;
  license: string;
  error: string | null;
  has_custom_content: boolean;
  custom_html?: string;
}

interface WikipediaContentProps {
  topicId: number;
  topicName: string;
}

export default function WikipediaContent({ topicId, topicName }:
WikipediaContentProps) {
  const [content, setContent] = useState<WikiContent | null>(null);
  const [loading, setLoading] = useState(true);
  const [showFull, setShowFull] = useState(false);
  const [error, setError] = useState('');

  useEffect(() => {
    setLoading(true);
    setContent(null);
    setShowFull(false);
    setError('');

    aptitudeAPI.getTopicWikiContent(topicId)
      .then(res => {
        setContent(res.data);
        if (!res.data.success && res.data.error) {
```

```
          setError(res.data.error);
        }
      })
      .catch(() => {
        setError('Could not load theory content. Please check your
connection.');
      })
      .finally(() => setLoading(false));

  }, [topicId]);

  if (loading) {
    return (
      <div className="animate-pulse space-y-3 p-4">
        <div className="h-4 bg-gray-200 rounded w-3/4"/>
        <div className="h-4 bg-gray-200 rounded w-full"/>
        <div className="h-4 bg-gray-200 rounded w-5/6"/>
        <div className="h-4 bg-gray-200 rounded w-full"/>
        <div className="h-4 bg-gray-200 rounded w-2/3"/>
      </div>
    );
  }

  return (
    <div className="space-y-4">

      {/* Custom HTML content from DB (if exists) — show first */}
      {content?.has_custom_content && content.custom_html && (
        <div
          className="prose max-w-none text-gray-800"
          dangerouslySetInnerHTML={{ __html: content.custom_html }}
        />
      )}

      {/* Wikipedia Theory Section */}
      {content?.success ? (
        <div className="border border-gray-200 rounded-xl overflow-hidden">

          {/* Header bar */}
          <div className="flex items-center gap-3 px-4 py-3 bg-gray-50
border-b border-gray-200">
            <img

src="https://upload.wikimedia.org/wikipedia/commons/thumb/8/80/Wikipedia-
logo-v2.svg/32px-Wikipedia-logo-v2.svg.png"
              alt="Wikipedia"
              className="w-6 h-6"
            />
            <span className="text-sm font-semibold text-gray-700">
              Theory — from Wikipedia
            </span>
            <span className="ml-auto text-xs text-gray-400">
              {content.license}
            </span>
          </div>

          <div className="p-5 space-y-4">

            {/* Description badge */}
            {content.description && (
```

```
              <p className="text-sm text-blue-700 bg-blue-50 px-3 py-2
rounded-lg font-medium">
                {content.description}
              </p>
            )}

            {/* Thumbnail + Summary side by side */}
            <div className="flex gap-4">
              {content.thumbnail_url && (
                <img
                  src={content.thumbnail_url}
                  alt={content.topic_name}
                  className="w-32 h-24 object-cover rounded-lg flex-shrink-
0"
                  onError={(e) => {
                    // Hide image if it fails to load
                    (e.target as HTMLImageElement).style.display = 'none';
                  }}
                />
              )}
              <div
                className="prose prose-sm max-w-none text-gray-700 flex-1"
                dangerouslySetInnerHTML={{ __html: content.summary_html ||
content.summary }}
              />
            </div>

            {/* Full content expandable section */}
            {content.full_content && content.full_content.length > 200 && (
              <>
                {showFull && (
                  <div className="text-sm text-gray-700 whitespace-pre-line
leading-relaxed border-t pt-4">
                    {content.full_content}
                  </div>
                )}
                <button
                  onClick={() => setShowFull(prev => !prev)}
                  className="text-sm text-blue-600 hover:text-blue-800
font-medium
                             flex items-center gap-1 transition-colors"
                >
                  {showFull ? '▲ Show less' : '▼ Read full theory'}
                </button>
              </>
            )}

            {/* Wikipedia attribution link */}
            <div className="pt-2 border-t border-gray-100">

                href={content.wikipedia_url}
                target="_blank"
                rel="noopener noreferrer"
                className="text-xs text-gray-400 hover:text-blue-600
transition-colors"
              >
                📖 Read full article on Wikipedia →
              </a>
            </div>
          </div>
        </div>
```

```
      ) : (
        /* Wikipedia fetch failed — show helpful message, not a crash */
        <div className="border border-amber-200 rounded-xl p-4 bg-amber-
50">
          <p className="text-sm text-amber-800 font-medium">
            Theory content is temporarily unavailable for this topic.
          </p>
          <p className="text-xs text-amber-600 mt-1">
            {error || 'Please try refreshing the page.'}
          </p>
          {content?.wikipedia_url && (

              href={content.wikipedia_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-blue-600 mt-2 inline-block
hover:underline"
            >
              Open Wikipedia directly →
            </a>
          )}
        </div>
      )}
    </div>
  );
}
```

PART 2 — YOUTUBE DATA API v3

2.1 What You Already Have

From your Google Cloud Console screenshot, the YouTube Data API v3 is already enabled on your project "My Project YouTube 60761" and the status shows "Enabled." You selected "None" for application restrictions during development, which is correct. You now just need to follow the complete integration steps below.

2.2 Understanding Your Quota

Your free quota is 10,000 units per day. One search request costs 100 units. This means you can make 100 search requests per day for free. Since your project serves students who all study the same aptitude topics, caching is the key strategy. After the first student searches for "Percentages" videos, the results are cached in your Django cache backend for 24 hours. Every other student who opens the Percentages topic that day gets the cached result instantly with zero API calls. In practice, your 100 searches per day will serve thousands of students.

The free $300 trial credit you saw in the banner also covers overages, but you will never reach overages with smart caching in place.

2.3 Complete Backend Service File

Create this file at:

```
backend/apps/aptitude/services/youtube_service.py
# backend/apps/aptitude/services/youtube_service.py
"""
YouTube Data API v3 Service for AI Pre-Placement Trainer
Fetches 10 relevant educational videos for each aptitude topic.
Uses Django cache backend to minimize API quota consumption.
Author: Based on project blueprint by Vedant Kumbhar, Meeraj Krishna,
Loukik Ingale
"""

import requests
import logging
from typing import List, Dict, Any, Optional
from django.conf import settings
from django.core.cache import cache

logger = logging.getLogger(__name__)

# Cache for 24 hours — videos for a topic are unlikely to change daily
CACHE_TIMEOUT = 86400

# Request timeout in seconds
REQUEST_TIMEOUT = 10

# YouTube API base URL
YOUTUBE_SEARCH_URL = "https://www.googleapis.com/youtube/v3/search"
YOUTUBE_VIDEOS_URL = "https://www.googleapis.com/youtube/v3/videos"


class YouTubeServiceError(Exception):
    """
    Custom exception for YouTube API failures.
    Contains a user-friendly message and the original error for logging.
    """
    def __init__(self, message: str, status_code: int = 503,
original_error: str = ""):
        self.message = message
        self.status_code = status_code
        self.original_error = original_error
        super().__init__(message)


class YouTubeService:
    """
    Service class wrapping YouTube Data API v3 search and video detail
endpoints.
    Handles quota awareness, caching, error handling, and response
formatting.
    """
```

```
    def __init__(self):
        self.api_key = getattr(settings, 'YOUTUBE_API_KEY', None)
        if not self.api_key:
            logger.error(
                "YOUTUBE_API_KEY is not set in settings/environment. "
                "YouTube video features will not work."
            )

    def _check_api_key(self):
        """Validate that the API key is configured before making
requests."""
        if not self.api_key:
            raise YouTubeServiceError(
                "YouTube API key is not configured. "
                "Please add YOUTUBE_API_KEY to your .env file.",
                status_code=500
            )

    def search_videos(
        self,
        topic_name: str,
        max_results: int = 10
    ) -> Dict[str, Any]:
        """
        Search YouTube for educational videos on a given aptitude topic.
        Results are cached for 24 hours to minimize API quota usage.

        Args:
            topic_name: The aptitude topic (e.g., "Percentages", "Time and
Work")
            max_results: Number of videos to return (default 10, max 10 per
quota limits)

        Returns:
            Dictionary with keys: topic_name, total, videos (list),
quota_note

        Raises:
            YouTubeServiceError: On API key missing, quota exceeded, or
network errors
        """
        self._check_api_key()

        max_results = min(max(max_results, 1), 10)

        # Build cache key from topic name
        cache_key = (
            f"yt_videos_{topic_name.replace(' ', '_').replace('/',
'_').lower()}"
        )

        cached = cache.get(cache_key)
        if cached:
            logger.info(f"YouTube cache HIT for topic: {topic_name}")
            cached["from_cache"] = True
            return cached

        logger.info(f"YouTube cache MISS for topic: {topic_name} — calling
API")
```

```
        # Build a search query optimized for Indian placement exam
preparation
        search_query = self._build_search_query(topic_name)

        params = {
            "part": "snippet",
            "q": search_query,
            "type": "video",
            "maxResults": max_results,
            "relevanceLanguage": "en",
            "videoEmbeddable": "true",     # Only embeddable videos
            "videoDuration": "medium",      # 4-20 minutes — best for
learning
            "videoDefinition": "any",
            "safeSearch": "strict",         # Educational content only
            "order": "relevance",
            "key": self.api_key,
        }

        try:
            response = requests.get(
                YOUTUBE_SEARCH_URL,
                params=params,
                timeout=REQUEST_TIMEOUT
            )

            # Handle YouTube-specific error codes
            if response.status_code == 403:
                error_data = response.json().get("error", {})
                error_message = error_data.get("message", "")
                errors = error_data.get("errors", [])
                reason = errors[0].get("reason", "") if errors else ""

                if reason == "quotaExceeded" or "quota" in
error_message.lower():
                    logger.error("YouTube API daily quota exceeded")
                    raise YouTubeServiceError(
                        "YouTube video quota has been reached for today. "
                        "Videos will be available again tomorrow. "
                        "Please use the Wikipedia content for now.",
                        status_code=429
                    )

                logger.error(
                    f"YouTube API 403 Forbidden. Reason: {reason}. "
                    f"Message: {error_message}"
                )
                raise YouTubeServiceError(
                    "YouTube API access denied. Please check the API key
configuration.",
                    status_code=403
                )

            if response.status_code == 400:
                logger.error(
                    f"YouTube API 400 Bad Request for topic: {topic_name}"
                )
                raise YouTubeServiceError(
                    "Invalid search request sent to YouTube.",
                    status_code=400
                )
```

```
            if response.status_code != 200:
                logger.error(
                    f"YouTube API returned unexpected status "
                    f"{response.status_code} for topic: {topic_name}"
                )
                raise YouTubeServiceError(
                    f"YouTube API error (status {response.status_code}). "
                    f"Please try again later.",
                    status_code=response.status_code
                )

            data = response.json()

            # Check if the response contains an error object
            if "error" in data:
                error_info = data["error"]
                logger.error(
                    f"YouTube API response error:
{error_info.get('message', 'Unknown')}"
                )
                raise YouTubeServiceError(
                    "YouTube returned an error response. Please try
again.",
                    status_code=error_info.get("code", 503)
                )

            items = data.get("items", [])

            if not items:
                logger.warning(
                    f"YouTube returned zero results for topic:
{topic_name}, "
                    f"query: {search_query}"
                )
                return {
                    "topic_name": topic_name,
                    "search_query": search_query,
                    "total": 0,
                    "videos": [],
                    "from_cache": False,
                    "quota_note": "10,000 units/day free",
                }

            # Format the video data
            videos = self._format_video_list(items)

            result = {
                "topic_name": topic_name,
                "search_query": search_query,
                "total": len(videos),
                "videos": videos,
                "from_cache": False,
                "quota_note": "10,000 units/day free — results cached
24hrs",
            }

            # Store in cache for 24 hours
            cache.set(cache_key, result, CACHE_TIMEOUT)

            return result
```

```
        except YouTubeServiceError:
            raise

        except requests.exceptions.Timeout:
            logger.error(f"YouTube API request timed out for topic:
{topic_name}")
            raise YouTubeServiceError(
                "YouTube is taking too long to respond. Please try again.",
                status_code=504
            )

        except requests.exceptions.ConnectionError:
            logger.error(
                f"Cannot connect to YouTube API for topic: {topic_name}"
            )
            raise YouTubeServiceError(
                "Cannot connect to YouTube. Please check your internet
connection.",
                status_code=503
            )

        except requests.exceptions.JSONDecodeError:
            logger.error(
                f"YouTube API returned invalid JSON for topic:
{topic_name}"
            )
            raise YouTubeServiceError(
                "YouTube returned an invalid response. Please try again.",
                status_code=503
            )

        except Exception as e:
            logger.error(
                f"Unexpected error in YouTube search for topic "
                f"'{topic_name}': {str(e)}"
            )
            raise YouTubeServiceError(
                "An unexpected error occurred while loading videos. "
                "Please try again later.",
                status_code=500,
                original_error=str(e)
            )

    def _build_search_query(self, topic_name: str) -> str:
        """
        Build an optimized YouTube search query for aptitude topics.
        Adds keywords that return high-quality educational Indian placement
content.

        Args:
            topic_name: Raw topic name from your database

        Returns:
            Optimized search query string
        """
        # Topics that need specific search terms to get the right content
        TOPIC_QUERY_MAP = {
            "Number Systems": "number system aptitude shortcuts tricks",
            "HCF and LCM": "HCF LCM aptitude tricks shortcuts explained",
```

```
            "Percentages": "percentage aptitude tricks shortcuts
placement",
            "Profit and Loss": "profit loss aptitude shortcuts tricks
placement",
            "Simple Interest": "simple interest aptitude formula
shortcuts",
            "Compound Interest": "compound interest aptitude tricks
formula",
            "Time and Work": "time and work aptitude shortcuts tricks",
            "Time Speed Distance": "time speed distance aptitude tricks
shortcuts",
            "Pipes and Cisterns": "pipes and cisterns aptitude tricks",
            "Ratio and Proportion": "ratio proportion aptitude shortcuts",
            "Averages": "averages aptitude tricks shortcuts placement",
            "Probability": "probability aptitude tricks shortcuts",
            "Permutations and Combinations": "permutation combination
aptitude tricks",
            "Logical Reasoning": "logical reasoning aptitude tricks
placement",
            "Coding and Decoding": "coding decoding reasoning tricks
shortcuts",
            "Blood Relations": "blood relations reasoning tricks
shortcuts",
            "Syllogism": "syllogism reasoning tricks shortcuts",
            "Data Interpretation": "data interpretation aptitude tricks
shortcuts",
        }

        if topic_name in TOPIC_QUERY_MAP:
            return TOPIC_QUERY_MAP[topic_name]

        # Default: append common aptitude keywords
        return f"{topic_name} aptitude tricks shortcuts explained placement
exam"

    def _format_video_list(self, items: list) -> List[Dict[str, Any]]:
        """
        Format raw YouTube API items into clean, consistent video objects.

        Args:
            items: Raw list of YouTube search result items

        Returns:
            List of formatted video dictionaries
        """
        videos = []

        for item in items:
            try:
                # Skip if this is not a video item
                if item.get("id", {}).get("kind") != "youtube#video":
                    continue

                video_id = item["id"].get("videoId", "")
                if not video_id:
                    continue

                snippet = item.get("snippet", {})
                thumbnails = snippet.get("thumbnails", {})

                # Pick the best available thumbnail quality
```

```
                thumbnail_url = (
                    thumbnails.get("medium", {}).get("url")
                    or thumbnails.get("default", {}).get("url")
                    or thumbnails.get("high", {}).get("url")
                    or ""
                )

                # Clean the title — remove excessive emoji and special
chars
                title = snippet.get("title", "").strip()

                # Format published date to readable form
                published_raw = snippet.get("publishedAt", "")
                published_date = published_raw[:10] if published_raw else
""

                # Truncate description to 150 characters
                description = snippet.get("description", "").strip()
                if len(description) > 150:
                    description = description[:147] + "..."

                videos.append({
                    "video_id": video_id,
                    "title": title,
                    "channel_name": snippet.get("channelTitle", ""),
                    "description": description,
                    "thumbnail_url": thumbnail_url,
                    "published_date": published_date,
                    "embed_url":
f"https://www.youtube.com/embed/{video_id}?rel=0&modestbranding=1",
                    "watch_url":
f"https://www.youtube.com/watch?v={video_id}",
                })

            except (KeyError, TypeError) as e:
                logger.warning(
                    f"Skipping malformed YouTube item due to error:
{str(e)}"
                )
                continue

        return videos

    def clear_topic_cache(self, topic_name: str) -> bool:
        """
        Manually clear the cached videos for a topic.
        Use this from Django admin when you want fresh results.

        Args:
            topic_name: Topic name whose cache should be cleared

        Returns:
            True if cache was cleared, False if no cache existed
        """
        cache_key = (
            f"yt_videos_{topic_name.replace(' ', '_').replace('/',
'_').lower()}"
        )
        if cache.get(cache_key):
            cache.delete(cache_key)
            logger.info(f"Cleared YouTube cache for topic: {topic_name}")
```

```
            return True

        logger.info(f"No YouTube cache found for topic: {topic_name}")
        return False
```

2.4 Complete Django View for YouTube Videos

Replace the basic view from the previous session with this fully error-handled version in `apps/aptitude/views.py`:

```
# Initialize the YouTube service at module level
from .services.youtube_service import YouTubeService, YouTubeServiceError

youtube_service = YouTubeService()


@api_view(['GET'])
@permission_classes([IsAuthenticated])
def get_topic_videos(request, topic_id):
    """
    Fetch 10 YouTube educational videos for a specific aptitude topic.
    Results are cached for 24 hours to minimize API quota consumption.

    URL: GET /api/aptitude/topics/{topic_id}/videos/

    Query params:
        count (optional): Number of videos, default 10, max 10

    Response on success (200):
    {
        "success": true,
        "topic_id": 3,
        "topic_name": "Percentages",
        "total": 10,
        "from_cache": false,
        "videos": [
            {
                "video_id": "abc123",
                "title": "Percentages Tricks and Shortcuts",
                "channel_name": "Unacademy Aptitude",
                "description": "Learn percentage shortcuts...",
                "thumbnail_url": "https://i.ytimg.com/vi/...",
                "published_date": "2023-06-15",
                "embed_url": "https://www.youtube.com/embed/abc123?rel=0",
                "watch_url": "https://www.youtube.com/watch?v=abc123"
            },
            ...
        ]
    }

    Response on quota exceeded (429):
    {
        "success": false,
        "error": "YouTube video quota has been reached for today...",
        "videos": [],
        "fallback_url": "https://www.youtube.com/results?search_query=..."
    }
    """
```

```
    try:
        topic = AptitudeTopic.objects.get(id=topic_id, is_active=True)
    except AptitudeTopic.DoesNotExist:
        return Response(
            {
                "success": False,
                "error": f"Topic with ID {topic_id} does not exist.",
                "videos": [],
            },
            status=404
        )

    max_results = int(request.query_params.get('count', 10))
    max_results = min(max(max_results, 1), 10)

    try:
        result = youtube_service.search_videos(
            topic_name=topic.name,
            max_results=max_results
        )

        return Response({
            "success": True,
            "topic_id": topic_id,
            "topic_name": topic.name,
            "total": result["total"],
            "from_cache": result.get("from_cache", False),
            "videos": result["videos"],
            "quota_note": result.get("quota_note", ""),
        }, status=200)

    except YouTubeServiceError as e:
        logger.warning(
            f"YouTubeServiceError for topic '{topic.name}': {e.message}"
        )

        # Build a fallback YouTube search URL
        # If the API fails, students can still search manually
        fallback_query = topic.name.replace(' ', '+') + '+aptitude+tricks'
        fallback_url =
f"https://www.youtube.com/results?search_query={fallback_query}"

        return Response({
            "success": False,
            "topic_id": topic_id,
            "topic_name": topic.name,
            "error": e.message,
            "videos": [],
            "fallback_url": fallback_url,
            "fallback_message": (
                f"You can search for '{topic.name} aptitude tricks' "
                f"directly on YouTube."
            ),
        }, status=200)  # Always 200 so the frontend does not crash
```

2.5 Add Settings and Install Dependencies

Add to `backend/.env`:

```
# YouTube Data API v3 key from Google Cloud Console
YOUTUBE_API_KEY=AIzaSy...your_key_here

# Django cache directory (creates a folder in your project root)
CACHE_DIR=./cache
```

Add to `ai_trainer/settings.py`:

```
from decouple import config

# YouTube API Key
YOUTUBE_API_KEY = config('YOUTUBE_API_KEY', default='')

# File-based cache — no Redis needed for development
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.filebased.FileBasedCache',
        'LOCATION': BASE_DIR / 'cache',
        'TIMEOUT': 86400,
        'OPTIONS': {
            'MAX_ENTRIES': 1000,
        }
    }
}
```

Create the cache directory:

```
mkdir backend/cache
```

Add to `requirements.txt`:

```
requests==2.31.0
```

2.6 Update Frontend API Service

Update `src/services/aptitudeAPI.ts` to include both new endpoints:

```
// src/services/aptitudeAPI.ts
import api from './api';

export const aptitudeAPI = {

  // Existing endpoints
  getTopics: () =>
    api.get('/aptitude/topics/'),

  getTopicDetail: (topicId: number) =>
    api.get(`/aptitude/topics/${topicId}/`),

  getQuestions: (topicId: number, count = 10) =>
    api.get(`/aptitude/questions/?topic_id=${topicId}&count=${count}`),

  submitQuiz: (topicId: number, answers: Record<string, string>) =>
    api.post('/aptitude/submit/', { topic_id: topicId, answers }),

  getQuizHistory: () =>
```

```
    api.get('/aptitude/history/'),

  // YouTube videos — returns 10 educational videos for the topic
  getTopicVideos: (topicId: number, count = 10) =>
    api.get(`/aptitude/topics/${topicId}/videos/?count=${count}`),

  // Wikipedia theory content — returns summary + full extract
  getTopicWikiContent: (topicId: number) =>
    api.get(`/aptitude/topics/${topicId}/wiki-content/`),

  // Wikipedia search — for the search box feature
  searchWikipedia: (query: string, limit = 5) =>
    api.get(`/aptitude/wiki-
search/?q=${encodeURIComponent(query)}&limit=${limit}`),
};
```

2.7 Complete Video Section Frontend Component

This is the complete, production-ready VideoSection component that shows all 10 videos with the main player and thumbnail list:

```
// src/components/learning/VideoSection.tsx
import { useState, useEffect, useCallback } from 'react';
import { aptitudeAPI } from '../../services/aptitudeAPI';

interface Video {
  video_id: string;
  title: string;
  channel_name: string;
  description: string;
  thumbnail_url: string;
  published_date: string;
  embed_url: string;
  watch_url: string;
}

interface VideoSectionProps {
  topicId: number;
  topicName: string;
}

type LoadState = 'loading' | 'success' | 'error' | 'quota';

export default function VideoSection({ topicId, topicName }:
VideoSectionProps) {
  const [videos, setVideos] = useState<Video[]>([]);
  const [activeVideo, setActiveVideo] = useState<Video | null>(null);
  const [loadState, setLoadState] = useState<LoadState>('loading');
  const [errorMessage, setErrorMessage] = useState('');
  const [fallbackUrl, setFallbackUrl] = useState('');
  const [fromCache, setFromCache] = useState(false);

  const loadVideos = useCallback(() => {
    setLoadState('loading');
    setVideos([]);
    setActiveVideo(null);
    setErrorMessage('');
    setFallbackUrl('');
```

```
    aptitudeAPI.getTopicVideos(topicId, 10)
      .then(res => {
        const data = res.data;

        if (data.success && data.videos.length > 0) {
          setVideos(data.videos);
          setActiveVideo(data.videos[0]);
          setFromCache(data.from_cache || false);
          setLoadState('success');
        } else if (!data.success) {
          // Check if it is a quota error
          if (data.error && data.error.toLowerCase().includes('quota')) {
            setLoadState('quota');
          } else {
            setLoadState('error');
          }
          setErrorMessage(data.error || 'No videos found for this topic.');
          setFallbackUrl(data.fallback_url || '');
        } else {
          // Success but zero results
          setLoadState('error');
          setErrorMessage(`No videos found for "${topicName}". Try
searching on YouTube directly.`);
        }
      })
      .catch(() => {
        setLoadState('error');
        setErrorMessage('Could not load videos. Please check your
connection.');
      });

  }, [topicId, topicName]);

  useEffect(() => {
    loadVideos();
  }, [loadVideos]);

  // Loading skeleton
  if (loadState === 'loading') {
    return (
      <div className="space-y-3">
        <div className="w-full h-56 bg-gray-200 rounded-xl animate-pulse"/>
        <div className="grid grid-cols-1 gap-2">
          {[1, 2, 3].map(i => (
            <div key={i} className="flex gap-3 animate-pulse">
              <div className="w-28 h-16 bg-gray-200 rounded-md flex-shrink-
0"/>
              <div className="flex-1 space-y-2">
                <div className="h-3 bg-gray-200 rounded w-full"/>
                <div className="h-3 bg-gray-200 rounded w-3/4"/>
              </div>
            </div>
          ))}
        </div>
      </div>
    );
  }

  // Quota exceeded state
  if (loadState === 'quota') {
```

```
    return (
      <div className="border border-amber-200 rounded-xl p-5 bg-amber-50
space-y-3">
        <div className="flex items-center gap-2">
          <span className="text-amber-600 text-xl">⚠□</span>
          <p className="font-semibold text-amber-800">Daily video limit
reached</p>
        </div>
        <p className="text-sm text-amber-700">{errorMessage}</p>
        {fallbackUrl && (

            href={fallbackUrl}
            target="_blank"
            rel="noopener noreferrer"
            className="inline-flex items-center gap-2 text-sm font-medium
text-red-600
                        hover:text-red-800 bg-white border border-red-200
                        px-4 py-2 rounded-lg transition-colors"
          >
            🎬 Search on YouTube directly →
          </a>
        )}
      </div>
    );
  }

  // Error state
  if (loadState === 'error') {
    return (
      <div className="border border-red-100 rounded-xl p-5 bg-red-50 space-
y-3">
        <p className="text-sm text-red-700 font-medium">{errorMessage}</p>
        <div className="flex gap-2">
          <button
            onClick={loadVideos}
            className="text-sm text-blue-600 hover:text-blue-800 font-
medium"
          >
            🔄 Try again
          </button>
          {fallbackUrl && (

              href={fallbackUrl}
              target="_blank"
              rel="noopener noreferrer"
              className="text-sm text-red-600 hover:text-red-800 font-
medium"
            >
              Open YouTube →
            </a>
          )}
        </div>
      </div>
    );
  }

  // Success state — full video section
  return (
    <div className="space-y-4">
```

```
      {/* Section header */}
      <div className="flex items-center justify-between">
        <h3 className="text-base font-semibold text-gray-800 flex items-
center gap-2">
          🎬 {videos.length} Video Lessons — {topicName}
        </h3>
        {fromCache && (
          <span className="text-xs text-gray-400 bg-gray-100 px-2 py-1
rounded-full">
            Cached
          </span>
        )}
      </div>

      {/* Main Video Player */}
      {activeVideo && (
        <div className="rounded-xl overflow-hidden shadow-md bg-black">
          <iframe
            key={activeVideo.video_id}
            src={activeVideo.embed_url}
            title={activeVideo.title}
            width="100%"
            height="360"
            frameBorder="0"
            allow="accelerometer; autoplay; clipboard-write; encrypted-
media; gyroscope; picture-in-picture; web-share"
            allowFullScreen
            className="block"
          />
          <div className="p-3 bg-gray-900 text-white">
            <p className="font-medium text-sm leading-
snug">{activeVideo.title}</p>
            <div className="flex items-center justify-between mt-1">
              <p className="text-xs text-gray-
400">{activeVideo.channel_name}</p>

                href={activeVideo.watch_url}
                target="_blank"
                rel="noopener noreferrer"
                className="text-xs text-red-400 hover:text-red-300"
              >
                Watch on YouTube ↗
              </a>
            </div>
          </div>
        </div>
      )}

      {/* Video Playlist — all 10 results */}
      <div className="border border-gray-200 rounded-xl overflow-hidden">
        <div className="px-3 py-2 bg-gray-50 border-b border-gray-200">
          <p className="text-xs text-gray-500 font-medium uppercase
tracking-wide">
            Playlist — {videos.length} videos
          </p>
        </div>

        <div className="divide-y divide-gray-100 max-h-80 overflow-y-auto">
          {videos.map((video, index) => (
            <div
              key={video.video_id}
```

```
                onClick={() => setActiveVideo(video)}
                className={`flex gap-3 p-3 cursor-pointer transition-colors
                  ${activeVideo?.video_id === video.video_id
                    ? 'bg-blue-50 border-l-4 border-l-blue-500'
                    : 'hover:bg-gray-50 border-l-4 border-l-transparent'
                  }`}
              >
                {/* Thumbnail with number badge */}
                <div className="relative flex-shrink-0">
                  <img
                    src={video.thumbnail_url}
                    alt={video.title}
                    className="w-28 h-16 object-cover rounded-md"
                    onError={(e) => {
                      // Replace broken thumbnail with placeholder
                      (e.target as HTMLImageElement).src =

'
PSIwIDAgMTEyIDYzIiBmaWxsPSJub25lIiB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAwMC9
zdmciPjxyZWN0IHdpZHRoPSIxMTIiIGhlaWdodD0iNjMiIGZpbGw9IiNGM0Y0RjYiLz48dGV4dC
B4PSI1NiIgeT0iMzUiIHRleHQtYW5jaG9yPSJtaWRkbGUiIGZpbGw9IiM5Q0EzQUYiIGZvbnQtc
2l6ZT0iMjAiPuKWtjwvdGV4dD48L3N2Zz4=';
                    }}
                  />
                  {/* Playing indicator overlay */}
                  {activeVideo?.video_id === video.video_id && (
                    <div className="absolute inset-0 bg-blue-600/20 rounded-
md
                                    flex items-center justify-center">
                      <div className="bg-blue-600 rounded-full w-6 h-6
                                      flex items-center justify-center">
                        <span className="text-white text-xs">▶</span>
                      </div>
                    </div>
                  )}
                  {/* Video number badge */}
                  <span className="absolute bottom-1 left-1 bg-black/70 text-
white
                                   text-xs px-1.5 py-0.5 rounded font-mono">
                    {String(index + 1).padStart(2, '0')}
                  </span>
                </div>

                {/* Video info */}
                <div className="flex-1 min-w-0">
                  <p className={`text-sm leading-snug line-clamp-2
                    ${activeVideo?.video_id === video.video_id
                      ? 'font-semibold text-blue-800'
                      : 'font-medium text-gray-800'
                    }`}>
                    {video.title}
                  </p>
                  <p className="text-xs text-gray-500 mt-1 truncate">
                    {video.channel_name}
                  </p>
                  <p className="text-xs text-gray-400 mt-0.5">
                    {video.published_date}
                  </p>
                </div>
              </div>
            ))}
```

```
        </div>
      </div>

      {/* Attribution note */}
      <p className="text-xs text-gray-400 text-center">
        Video content provided by YouTube creators. Content belongs to
respective channel owners.
      </p>
    </div>
  );
}
```

2.8 Wire Everything Together in TopicContent.tsx

This is the final integration step. Your existing `TopicContent.tsx` renders the selected topic. Add both new sections to it:

```
// src/components/learning/TopicContent.tsx  (updated)
import WikipediaContent from './WikipediaContent';
import VideoSection from './VideoSection';

export default function TopicContent({ topic }) {

  if (!topic) {
    return (
      <div className="flex items-center justify-center h-64 text-gray-400">
        Select a topic from the sidebar to start learning
      </div>
    );
  }

  return (
    <div className="space-y-8 pb-12">

      {/* Topic Header */}
      <div className="border-b pb-4">
        <h1 className="text-2xl font-bold text-gray-900">{topic.name}</h1>
        {topic.description && (
          <p className="text-gray-500 mt-1 text-sm">{topic.description}</p>
        )}
      </div>

      {/* Section 1 — Theory from Wikipedia + custom DB content */}
      <section>
        <h2 className="text-lg font-semibold text-gray-800 mb-3 flex items-
center gap-2">
          📖 Theory
        </h2>
        <WikipediaContent topicId={topic.id} topicName={topic.name} />
      </section>

      {/* Section 2 — 10 YouTube Video Lessons */}
      <section>
        <h2 className="text-lg font-semibold text-gray-800 mb-3 flex items-
center gap-2">
          🎬 Video Lessons
        </h2>
```

```
        <VideoSection topicId={topic.id} topicName={topic.name} />
      </section>

    </div>
  );
}
```

2.9 Final Checklist Before Testing

Go through each item below and verify it is done before running the application:

Backend checklist:

- `YOUTUBE_API_KEY` is set in `backend/.env`
- `backend/cache/` folder exists (run `mkdir backend/cache`)
- `CACHES` block is in `settings.py`
- `YOUTUBE_API_KEY = config(...)` is in `settings.py`
- `youtube_service.py` is saved at `apps/aptitude/services/youtube_service.py`
- `wikipedia_service.py` is saved at `apps/aptitude/services/wikipedia_service.py`
- `wikipedia_mapping.py` is saved at `apps/aptitude/wikipedia_mapping.py`
- Both new views are added to `apps/aptitude/views.py`
- Both new URL patterns are added to `apps/aptitude/urls.py`
- `requests` is in `requirements.txt` and installed via `pip install requests`

Frontend checklist:

- `aptitudeAPI.ts` has `getTopicVideos` and `getTopicWikiContent` methods
- `VideoSection.tsx` is saved at `src/components/learning/VideoSection.tsx`
- `WikipediaContent.tsx` is saved at `src/components/learning/WikipediaContent.tsx`
- `TopicContent.tsx` imports and renders both new components

Test the endpoints manually using your browser or Postman:

Wikipedia test (no API key needed):

```
GET http://localhost:8000/api/aptitude/topics/1/wiki-content/
Authorization: Bearer your_jwt_token_here
```

YouTube test (needs API key):

```
GET http://localhost:8000/api/aptitude/topics/1/videos/
Authorization: Bearer your_jwt_token_here
```

End of Guide — AI Pre-Placement Trainer API Integration Wikipedia API + YouTube Data API v3 — Complete Implementation YSPM's Yashoda Technical Campus, Satara | AI & Data Science | 2025-26